# 3 EF Core: sorğudan loading-ə addım-addım

Təsəvvür et ki, kafe tətbiqində Ayselin sifarişini və sifarişdəki məhsulları göstərmək istəyirsən. Proqram bazadan nə istəyir? Baza bunu nə vaxt oxuyur? Gələn nəticə C# obyektinə necə çevrilir? Məhsullar hansı anda gəlir?

Bu sualları eyni anda cavablandırmağa çalışmayacağıq. Əvvəl bir sifariş sorğusunu izləyəcəyik, sonra əlaqəli məhsulları əlavə edəcəyik.

**Birinci hissə:** bazada nə var, C#-da onu necə təmsil edirik, sorğunu necə qururuq və nə vaxt icra edirik.

**İkinci hissə:** IQueryable və IEnumerable fərqi. Eyni Where yazılışı hansı halda SQL-ə gedir, hansı halda C#-da işləyir? AsEnumerable və ToList nəyi dəyişir?

**Üçüncü hissə:** eager, explicit və lazy loading. Hər birində SQL-i başladan dəqiq sətri görəcəyik.

**Dördüncü hissə:** çoxlu sifariş, əlavə SQL-lər, tracking, filtrli Include, split query və işlək layihə.

Kodların əsas nəticələri .NET 10, EF Core 10.0.9 və real SQLite bazasında yoxlanılıb. Hər təcrübə təzə DbContext və eyni məlumatla başlayır. Başlanğıc məlumatını bazaya yazan əmrlər göstərilən SQL sayına daxil deyil.

Oxuyarkən hər kod üçün əvvəlcə bu iki sualı ver: **bu sətirdə SQL işlədimi? Nəticədə hansı məlumat artıq proqramdadır?**

# 1. Əvvəl bazada nə olduğunu bilək

**Baza** tətbiqin məlumatlarını saxladığı yerdir. **Cədvəl** eyni quruluşlu qeydləri bir yerə toplayır. Hər qeyd sətir, Id və ad kimi məlumat hissələri sütun adlanır.

Bizim bazada sifarişlə məhsul sətirləri ayrı cədvəllərdədir:

| Orders: Id | CustomerName |
|---|---|
| 42 | Aysel |
| 43 | Murad |
| 44 | Leyla |

| OrderItems: Id | OrderId | ProductName |
|---|---|---|
| 101 | 42 | Cay |
| 102 | 42 | Tort |
| 103 | 43 | Qehve |

Orders sifarişləri, OrderItems isə sifarişlərin məhsul sətirlərini saxlayır. OrderItems-dəki OrderId məhsulun hansı sifarişə aid olduğunu göstərir.

**Id əsas açardır:** cədvəldə sətri fərqləndirir. **OrderId xarici açardır:** başqa cədvəldəki əlaqəli sətrin Id-sini saxlayır. Burada OrderId = 42 olan iki sətir Ayselin sifarişinə aiddir.

42-nin iki, 43-ün bir məhsulu var. 44-ün məhsulu yoxdur. Bir sifarişdə çoxlu məhsul ola bildiyi üçün bu əlaqəyə **birdən-çoxa əlaqə** deyirik.

İndi bildiyimiz fakt budur: **bazada 42 üçün iki məhsul var.** Onların C# proqramına gəlib-gəlməməsi ayrıca məsələdir.

# 2. C# obyektləri bazadakı sətirləri təmsil edir

**Sinif - class** obyektin quruluşunu təsvir edir. **Obyekt** həmin quruluşdan yaranmış konkret nüsxədir. Məsələn, Order sinifdir, Ayselin 42 nömrəli sifarişini təmsil edən order isə obyektdir.

C# - modelimizin iki sinfi

```
public class Order
{
    public int Id { get; set; }
    public string CustomerName { get; set; } = "";
    public virtual ICollection<OrderItem> Items { get; set; }
        = new List<OrderItem>();
}

public class OrderItem
{
    public int Id { get; set; }
    public int OrderId { get; set; }
    public string ProductName { get; set; } = "";
    public virtual Order Order { get; set; } = null!;
}
```

**Property - xassə** obyektin Id kimi oxunan və ya dəyişən məlumat hissəsidir. EF-in baza modelində tanıdığı belə obyektə **entity** deyirik.

`OrderId` nömrədir. `Order` isə əlaqəli sifariş obyektinə keçiddir. `Items` sifarişə aid məhsul obyektlərinə keçiddir. Belə keçidlər **navigation property** adlanır.

`ICollection<OrderItem>` məhsul obyektlərinin toplusu üçün müqavilədir. `new List<OrderItem>()` boş siyahı yaradır. **Bu sətir bazadan məhsul oxumur.** `virtual` və `null!` yazılışını lazy loading və tam layihə hissəsində açacağıq.

# 3. EF Core və DbContext hansı işi görür?

**EF Core** C# obyektləri ilə baza cədvəlləri arasında işləməyə kömək edən kitabxanadır. Buna ORM də deyilir: obyektləri cədvəllərə uyğunlaşdıran vasitə.

**SQL** bazaya hansı məlumatı oxumağı və ya dəyişməyi bildirmək üçün istifadə olunan dildir. Biz C#-da sorğu yazırıq, EF Core uyğun hissələri SQL-ə çevirir.

**DbContext** EF-in sorğularını və dəyişikliklərin saxlanmasını idarə edən obyektdir. Dərsdə bu obyektin dəyişən adını `db` qoyuruq. **DbSet<Order>** isə Order-lar üçün sorğu yazmağa çıxış verir.

C# - db.Orders sorğunun başlanğıcıdır

```
var query = db.Orders.Where(o => o.Id == 42);
```

`db.Orders` bazadakı sifarişlərin artıq hamısının proqramda olması demək deyil. Bu sətirdə hələ sifarişləri oxumamışıq.

**LINQ** C#-da Where, Select, OrderBy kimi metodlarla sorğu yazmaq imkanının adıdır. Where şərtə uyğun olanları seçir. `o => o.Id == 42` burada “baxılan sifarişin Id-si 42-yə bərabərdir” şərtini ifadə edir. `o` müvəqqəti dəyişən adıdır, `==` bərabərlik yoxlamasıdır.

**Provider** EF-i seçilmiş baza ilə əlaqələndirən hissədir. Dərsdə SQLite provider-i var. SQLite ayrıca server proqramı tələb etmədən işləyən baza mühərrikidir. SQL Server-də SQL yazılışı fərqlənə bilər.

Hələlik bu yolu yadda saxla: **C# sorğusu -> EF və provider -> SQL -> baza nəticəsi.**

# 4. Sorğunu yazmaq hələ onu icra etmək deyil

**Query - sorğu** hansı məlumatı istədiyimizin təsviridir. **Execution - icra** həmin sorğunun işlədilib nəticəsinin alınmasıdır. Aşağıdakı iki sətir fərqli iş görür:

C# - bu iki sətri ayrı-ayrı oxu

```
var query = db.Orders.Where(o => o.Id == 42);

var orders = await query.ToListAsync();
```

Birinci sətir: “Orders-dən yalnız Id-si 42 olanları istəyəcəyəm” qaydasını qurur. Bu mərhələdə **SQL sayı 0-dır**.

İkinci sətir: “İndi həmin qaydanın nəticəsini gətir və siyahı hazırla” deyir. Bu nümunədə **bir SQL** icra olunur. `orders` artıq içində Order obyekti olan List-dir.

SQL - yaranan əmrin sadələşdirilmiş mənası

```
SELECT Id, CustomerName
FROM Orders
WHERE Id = 42;
```

`SELECT` gətiriləcək sütunları, `FROM` cədvəli, `WHERE` şərti bildirir. Provider-in faktiki əmri dırnaqlar və əlavə hissələr baxımından fərqlənə bilər.

**Materialization** gələn baza nəticəsindən C# obyektlərinin hazırlanmasıdır: EF nəticədəki 42 və Aysel dəyərlərini Order obyektinin xassələrinə yerləşdirir.

`ToListAsync` nəticəni siyahıya yığır. Async asinxron əməliyyatı bildirir; `await` nəticəni gözləyib sonra davam etməyə imkan verir. Async ayrıca loading növü deyil.

# 5. Deferred execution nə deməkdir?

**Deferred execution - təxirə salınmış icra** sorğunu qurarkən nəticənin hələ oxunmaması, oxuma işinin nəticəyə ehtiyac yaranan ana saxlanmasıdır.

C# - dotnet run -- foreach

```
var query = db.Orders.Where(o => o.Id == 42);
Console.WriteLine("Evvel SQL: " + sql.Count);

foreach (var order in query)
    Console.WriteLine("Id: " + order.Id);

Console.WriteLine("Sonra SQL: " + sql.Count);
```

Yoxlanmış çıxış

```
Evvel SQL: 0
Id: 42
Sonra SQL: 1
```

`sql` işlək layihədə icra olunmuş SQL əmrlərinin qeydlərini saxlayan siyahıdır. `sql.Count` həmin qeydlərin sayıdır.

`foreach` elementləri bir-bir oxuyan dövrdür. Sorğunun elementlərini oxumaq başlayanda bazaya müraciət edilir. Deməli, **SQL yalnız ToList ilə işləmir; foreach də EF sorğusunu icra edə bilər.**

`Where` və `OrderBy` kimi addımlar EF sorğusunu hazırlayır. `ToListAsync`, `FirstAsync`, `SingleAsync`, `CountAsync`, `AnyAsync` kimi nəticə istəyən əməliyyatlar onu icra edir.

Sorğu obyektinin mövcud olması nəticənin artıq gətirildiyini göstərmir. Sonrakı səhifələrdə məhz bu fərqi qoruyacağıq.

# 6. IQueryable nədir və bizə nə verir?

`IQueryable<Order>` Order sorğusu ilə işləmək üçün **interface**, yəni tipin təmin etdiyi imkanları bildirən müqavilədir. Buradakı `<Order>` sorğunun hansı tip elementlər üzərində qurulduğunu göstərir.

EF sorğusunu bu tipdə saxlayanda sonrakı uyğun Where kimi addımlar da bazaya göndəriləcək sorğunun təsvirinə əlavə oluna bilir:

C# - dotnet run -- query

```
IQueryable<Order> query = db.Orders
    .Where(o => o.Id >= 43);

query = query.Where(o => o.Id == 43);
var rows = await query.ToListAsync();
```

Birinci şərt 43 və 44-ü uyğun sayır. İkinci şərt bunlardan yalnız 43-ü saxlayır. İcra anında hər iki şərt SQL-ə daxil olur. **Bazadan bir Order sətri gəlir: Muradın 43 nömrəli sifarişi.**

EF bu şərtlərin strukturunu saxlayır. Buna **expression tree - ifadə ağacı** deyilir: “Id sütununu 43-lə müqayisə et” kimi əməliyyatların proqram daxilində təsviri. Provider bu təsviri oxuyub uyğun SQL hazırlayır.

IQueryable özü “SQL bazası” demək deyil. Onun davranışını arxasındakı provider müəyyən edir. Bizim nümunədə provider EF Core-un SQLite provider-idir; buna görə uyğun şərtlər SQL-ə çevrilir.

**İki Where yazdıq, amma iki SQL işləmədi.** Sorğunu iki addımda qurub bir dəfə icra etdik.

# 7. IEnumerable nədir? Hazır siyahı deməkdirmi?

`IEnumerable<Order>` “bu mənbənin Order elementlərini bir-bir oxuya bilərsən” müqaviləsidir. Elementləri oxumağa **enumeration** deyilir. Adi foreach bu imkandan istifadə edir.

Bu müqavilə elementlərin haradan gələcəyini demir. Hazır List-dən də gələ bilər, hələ icra edilməmiş EF sorğusundan da. Buna görə **IEnumerable yazılması məlumatın artıq yaddaşda olması demək deyil.**

C# - burada hələ SQL sayı 0-dır

```
IEnumerable<Order> source = db.Orders;
```

Yalnız dəyişənin görünən tipi dəyişdi. Əsl mənbə hələ EF sorğusudur. `foreach` və ya `ToList()` ilə oxunanda bazaya müraciət edilə bilər.

IQueryable<T> həm də IEnumerable<T>-dir. Yəni sorğu qurmaq imkanı olan mənbənin elementlərini oxumaq imkanı da var. Əks istiqamət ümumi qayda deyil: hər IEnumerable SQL qura bilmir.

C# - hazır List nümunəsi

```
List<int> numbers = new() { 1, 2, 3 };
IEnumerable<int> source = numbers;
```

Bu ikinci nümunədə elementlər artıq List-dədir. Heç bir baza yoxdur. IEnumerable hər iki vəziyyətə uyğun gəldiyindən yalnız tipin adına baxıb “SQL işlənib” nəticəsini çıxara bilmərik.

**List<T>** elementləri saxlayan konkret siyahı tipidir. **IEnumerable<T>** isə onları oxumaq imkanını bildirir. Bu fərq sonrakı Where-in harada işləyəcəyinə təsir edir.

# 8. Eyni Where niyə fərqli yerdə işləyir?

İki nümunənin də yekun nəticəsi yalnız 43-dür. Amma bazadan gələn məlumatın miqdarı fərqlənir. Hər variant ayrıca təzə context-də işlədilir.

C# - IQueryable: hər iki şərt SQL-ə gedir

```
IQueryable<Order> query = db.Orders
    .Where(o => o.Id >= 43);
var rows = await query.Where(o => o.Id == 43)
    .ToListAsync();
```

C# - IEnumerable: ikinci şərt C#-da işləyir

```
IEnumerable<Order> source = db.Orders
    .Where(o => o.Id >= 43);
var rows = source.Where(o => o.Id == 43).ToList();
```

Birincidə SQL həm `Id >= 43`, həm `Id == 43` şərtini daşıyır. Baza 43-ü qaytarır. **Bir Order obyekti alınır.**

İkincidə ilk Where tip dəyişməzdən əvvəl yazılıb və SQL-də qalır. Sonrakı Where IEnumerable üçün seçilir: gələn obyektlər C# kodu ilə yoxlanır. **Baza 43 və 44-ü qaytarır; C# 44-ü kənarlaşdırır.**

| Ölçü | IQueryable variantı | IEnumerable variantı |
|---|---|---|
| İcra olunmuş SQL | 1 | 1 |
| Oxunmuş Order obyektləri | 1 | 2 |
| Son siyahıdakı Id | 43 | 43 |

Eyni metod adı müxtəlif müqavilələr üçün təklif olunur. C# hansı variantı seçəcəyinə dəyişənin kodda görünən tipinə görə qərar verir. Burada ikinci variantın Where-i SQL təsviri qurmur; obyekt üzərində işləyən C# yoxlaması olur.

# 9. AsEnumerable nəyi dəyişir?

`AsEnumerable()` mənbəni IEnumerable kimi davam etdirməyə imkan verir. **Özü SQL icra etmir, özü siyahı yaratmır.** Əsas təsiri sonrakı LINQ addımlarının hansı qayda ilə seçilməsidir.

C# - dotnet run -- boundary

```
var query = db.Orders
    .Where(o => o.Id >= 43)
    .AsEnumerable()
    .Where(o => o.Id == 43);

// Buraya qədər SQL: 0
var rows = query.ToList();
// Buradan sonra SQL: 1
```

AsEnumerable-dan əvvəlki Where EF sorğusunun hissəsidir. Ona görə baza 43 və 44-ü göndərir. Ondan sonrakı Where obyektləri C#-da süzür; son siyahıda 43 qalır.

Məlumatın yolu

```
Baza: Id >= 43 -> 43 ve 44
Proqram: Id == 43 -> yalniz 43
```

Bu nümunədə axırda ToList yazdığımız üçün oxuma tamamlanır və siyahı yaranır. AsEnumerable özü bu işi görmürdü.

**Server-side** burada şərtin baza tərəfindən hesablanmasıdır. **Client-side** şərtin EF-dən istifadə edən C# proqramında hesablanmasıdır. Web tətbiqində həmin C# proqramı serverdə işləyə bilər; bu terminlərdəki client sözü brauzer demək deyil.

Məqsəd lazımsız sətirləri gətirməməkdirsə, bazaya çevrilə bilən daraldıcı şərtləri bu keçiddən əvvəl yaz.

# 10. ToList ilə AsEnumerable eyni deyil

C# - SQL işləyir, nəticə siyahıda saxlanır

```
var list = await db.Orders.ToListAsync();
var filtered = list.Where(o => o.Id == 43);
```

Birinci sətirdə üç sifariş də gətirilir. İkinci sətir siyahıdakı obyektlər üzərində filtr hazırlayır. `filtered` oxunanda filtr işləyir, amma Orders üçün yeni SQL lazım olmur.

| Əməliyyat | EF sorğusuna bu anda təsiri |
|---|---|
| Where | Uyğun şərti sorğuya əlavə edir; nəticə oxunmur. |
| AsEnumerable | Davamı IEnumerable kimi olur; nəticə hələ oxunmaya bilər. |
| ToList / ToListAsync | Mənbə oxunur, nəticələr List-də saxlanır. |
| foreach | Mənbənin elementləri oxunur; List yaratmaq məcburi deyil. |

Nəticələrin hamısını toplayıb saxlamağa **buffering** deyilir. ToList bunu edir. Elementləri tam siyahı tələb etmədən oxumağa **streaming** deyilir. EF sorğusunu foreach ilə oxumaq belə işləyə bilər.

Streaming “heç yaddaş işlənmir” demək deyil. Tracking və bəzi icra ayarları yenə obyektləri və ya nəticələri saxlaya bilər. Burada ayırd etdiyimiz şey proqramın açıq şəkildə tam List yaradıb-yaratmamasıdır.

**ToList bir icra və saxlama addımıdır. AsEnumerable isə sonrakı addımlar üçün müqaviləni dəyişir.**

# 11. Deferred yalnız IQueryable-a aid deyil

Hazır List üzərindəki Where də yoxlamanı nəticə oxunana qədər saxlaya bilər. Bu dəfə heç baza yoxdur:

C# - dotnet run -- list-deferred

```
var numbers = new List<int> { 1, 2 };
IEnumerable<int> query = numbers.Where(n => n > 1);

numbers.Add(3);
Console.WriteLine(string.Join(",", query));
```

Yoxlanmış nəticə

```
2,3
```

Where yazılan anda siyahıda 1 və 2 var idi. Lakin yoxlama hələ yerinə yetirilməmişdi. Sonra 3 əlavə olundu. `string.Join` nəticəni mətnə çevirmək üçün elementləri oxuyanda Where artıq 3-ü də gördü.

Deməli, **IEnumerable həmişə immediate, IQueryable həmişə deferred** kimi əzbərləmək yanlışdır. **Immediate execution** əməliyyat çağırışı zamanı nəticənin hesablanmasıdır; deferred isə nəticənin oxunmasını sonraya saxlayır.

Hazır siyahının özü artıq mövcud məlumatdır. Onun üzərində qurulan yeni Where nəticəsi isə hələ hesablanmamış ola bilər. Mənbənin hazır olması ilə yeni sorğunun icra olunması ayrı hadisələrdir.

Əgər `var result = numbers.Where(n => n > 1).ToList();` yazıb yalnız bundan sonra 3 əlavə etsəydik, result-da yalnız 2 qalardı. Çünki siyahı əvvəlki anda hazırlanmışdı.

# 12. Sorğunu iki dəfə oxusan nə baş verir?

EF sorğusunun nəticəsi avtomatik olaraq sorğu dəyişəninin içində saxlanmır. Eyni sorğunu yenidən oxumaq bazaya yenidən müraciət edə bilər.

C# - dotnet run -- repeat

```
var query = db.Orders.Where(o => o.Id >= 43);

foreach (var order in query) { }
Console.WriteLine("Ilk dovr SQL: " + sql.Count);

foreach (var order in query) { }
Console.WriteLine("Ikinci dovr SQL: " + sql.Count);

var list = await query.ToListAsync();
foreach (var order in list) { }
foreach (var order in list) { }
Console.WriteLine("List dovrleri SQL: " + sql.Count);
```

Yoxlanmış çıxış

```
Ilk dovr SQL: 1
Ikinci dovr SQL: 2
List dovrleri SQL: 3
```

İlk iki dövr iki SQL işlətdi. ToListAsync üçüncü SQL-i işlətdi və nəticəni saxladı. Son iki dövr hazır List-i oxudu; əlavə SQL yaratmadı.

Bu, **Orders sorğusuna** aiddir. Sonradan lazy loading aktiv obyektin yüklənməmiş əlaqəsini oxusan, o əlaqə ayrıca SQL yarada bilər. Hazır List yalnız əsas sorğunun nəticəsini saxlayır; bütün əlaqələrin də yükləndiyinə zəmanət vermir.

EF-in eyni SQL quruluşu üçün hazırlığı təkrar istifadə etməsi də nəticələrin yaddaşda saxlanması ilə eyni deyil.

# 13. Hər icra obyekt siyahısı gətirmir

Sənə neçə sifariş olduğunu bilmək lazımdırsa, bütün sifariş obyektlərini gətirmək vacib deyil. **Scalar nəticə** siyahı əvəzinə bir say və ya True/False kimi tək dəyərdir.

C# - dotnet run -- scalar

```
var query = db.Orders.Where(o => o.Id >= 43);

var count = await query.CountAsync();
var exists = await query.AnyAsync();
```

Bu sınaqda count = 2, exists = True olur. Hər əməliyyat bir SQL işlədir; birlikdə iki SQL-dir. Count SQL-də say hesablayır. Any ən azı bir uyğun sətir olub-olmadığını yoxlayır. **Order obyektləri yaradılmır; alınan nəticə say və məntiqi dəyərdir.**

| Metod | Gözlədiyi nəticə |
|---|---|
| ToListAsync | Uyğun elementlərdən siyahı. |
| FirstAsync | İlk uyğun element; heç biri yoxdursa xəta. |
| FirstOrDefaultAsync | İlk element; Order yoxdursa null. |
| SingleAsync | Dəqiq bir element; sıfır və ya çoxdursa xəta. |
| SingleOrDefaultAsync | Sıfırdırsa null; çoxdursa xəta. |
| CountAsync / AnyAsync | Say / uyğun elementin olub-olmaması. |

`null` burada obyekt nəticəsinin olmadığını bildirir. “İlk” nəticənin hansı olacağı vacibdirsə `OrderBy`, yəni sıralama yaz. Məsələn, `OrderBy(o => o.Id)` Id-yə görə artan sıra yaradır.

**Execution baş verdi deyə mütləq List və ya entity yaranmalı deyil.**

# 14. ToQueryString və AsQueryable tələləri

`ToQueryString()` EF sorğusu üçün nəzərdə tutulan SQL-in mətnini göstərmək üçündür. **Bazadan həmin sorğunun nəticəsini almır.** Query rejimində onun önündə də, arxasında da SQL sayğacı 0 qalır:

C# - query rejimindən

```
Console.WriteLine(query.ToQueryString());
var rows = await query.ToListAsync();
```

İkinci sətir real nəticəni gətirir. Faktiki icra olunan əmrləri görmək üçün isə layihədəki SQL qeydlərinə baxırıq. Xüsusilə split query-də bir SQL mətni bütün icra əmrlərini göstərməyə bilər.

`AsQueryable()` üçün də mənbənin nə olduğunu yoxla:

C# - dotnet run -- asqueryable

```
var list = await db.Orders.ToListAsync();
var query = list.AsQueryable().Where(o => o.Id == 43);
Console.WriteLine(query.Count());
```

Nəticə 1-dir, amma baza yalnız əvvəlki ToListAsync zamanı işlənib və üç sifariş də gətirilib. **Hazır List-ə AsQueryable yazmaq onu yenidən EF baza sorğusuna çevirmir.** Bu mənbə üçün hesablamalar yerli obyektlər üzərində qalır.

IQueryable adı təkbaşına SQL-ə zəmanət vermirdi; provider də vacib idi. Bu nümunə həmin qaydanın praktik nəticəsidir.

Burada List-ə çevrilmiş mənbədən danışırıq. Hələ EF sorğusu olan obyektin tipini geri dəyişməklə bağlı vəziyyətləri bu nümunə ilə eyniləşdirmə.

# 15. Hər C# metodunu SQL-ə çevirmək olmur

Tutaq ki, öz yoxlama metodumuzu yazmışıq:

C# - QueryLessons sinfindəki köməkçi metod

```
private static bool IsMurad(string name) => name == "Murad";
```

`bool` True və ya False dəyəridir. Bu metod ad Murad olduqda True qaytarır. Onu birbaşa EF Where-inə yazsaq:

C# - dotnet run -- translation

```
var rows = await db.Orders
    .Where(o => IsMurad(o.CustomerName))
    .ToListAsync();
```

Bu adi C# metodu üçün SQL çevirməsi təyin edilməyib. Sınaqda EF SQL göndərmədən çevirmə xətası verir. EF Where daxilindəki belə yoxlamanı gizlicə bütün cədvəli gətirərək həll etmir.

Məqsəd sadəcə Muradı seçməkdirsə birbaşa `Where(o => o.CustomerName == "Murad")` yazmaq bazada süzməyə imkan verir. Həqiqətən yalnız C#-da işləyən qayda lazımdırsa keçidi açıq yazırıq:

C# - dotnet run -- local

```
var rows = db.Orders.Where(o => o.Id >= 43)
    .AsEnumerable()
    .Where(o => IsMurad(o.CustomerName))
    .ToList();
```

Burada əvvəl bazada 43 və 44 seçilir, sonra C# metodu 43-ü saxlayır. Bu həll artıq məlum davranışdır; çox məlumat gətirməmək üçün əvvəlki baza filtrini düşünmək lazımdır. EF son Select-də bəzi yerli hesablamalara icazə verə bilər; Where barədəki qaydanı bütün ifadələrə tətbiq etmə.

# 16. Async bu fərqləri dəyişirmi?

`ToList()` nəticəni sinxron, `ToListAsync()` asinxron almaq üçündür. Async gözləmə zamanı icra axınını məşğul saxlamamaq imkanı verir; nəticənin mütləq daha tez gəlməsi demək deyil. Provider-in dəstəyi də rol oynayır.

**WhereAsync yazmırıq**, çünki adi EF Where nəticəni oxumur, sorğunu qurur. ToListAsync isə bazadan oxuma əməliyyatını başladır.

AsEnumerable-dan sonra adi IEnumerable LINQ metodları ilə davam edirik. Əvvəlki nümunədəki son ToList buna görə sinxrondur. Asinxron şəkildə elementləri oxumaq üçün başqa müqavilə var:

C# - dotnet run -- async-stream

```
var query = db.Orders.Where(o => o.Id >= 43)
    .AsAsyncEnumerable();

await foreach (var order in query)
    Console.WriteLine("Id: " + order.Id);
```

**IAsyncEnumerable<Order>** elementləri asinxron oxumağa imkan verir. `await foreach` həmin oxunuşları gözləyir. Bu sınaqda qurma zamanı SQL 0, dövr tamamlananda 1-dir; 43 və 44 oxunur.

Sorğu hələ oxunmamışsa, onun işləməsi üçün DbContext istifadəyə açıq qalmalıdır. Context-i bağlayıb sonra onun sorğusunu oxumağa çalışma.

Eyni DbContext-də paralel baza əməliyyatları başlatma; əvvəlkini await ilə tamamla. Lazy loading-də isə navigation oxunuşunun asinxron olub-olmadığını ayrıca görəcəyik.

# 17. İndi loading sualına keçirik

İndiyədək sorğunun **nə vaxt** icra olunduğunu və filtrin **harada** işlədiyini öyrəndik. İndi başqa sual var: **Order gələndə onun Items məlumatı da gəlirmi?**

Bu dərsdə loading əlaqəli məlumatın gətirilməsidir. Üç üsul həmin məlumatı hansı addımda istədiyimizi fərqli şəkildə ifadə edir:

| Üsul | Məhsulların gətirilməsi necə tələb olunur? |
|---|---|
| Eager loading | Order sorğusuna əvvəlcədən Include əlavə edirik. |
| Explicit loading | Order gələndən sonra ayrıca LoadAsync əmri yazırıq. |
| Lazy loading | Mexanizm aktivdirsə yüklənməmiş Items oxunuşu yükləməni başladır. |

Eager “bu kod sətrini yazan kimi SQL işləyir” demək deyil. Include ilə qurulmuş sorğu da ToListAsync və ya SingleAsync kimi icra addımını gözləyə bilər.

C# - hələ SQL yoxdur

```
var query = db.Orders.Include(o => o.Items);

// İcra burada başlayır.
var order = await query.SingleAsync(o => o.Id == 42);
```

**Deferred execution əsas sorğunun icra anıdır. Lazy loading isə alınmış obyektin əlaqəsini sonradan gətirmək mexanizmidir.** Hər ikisində gecikmə ola bilər, amma gecikdirilən iş eyni deyil.

Sonrakı nümunələrdə lazy yalnız lazy sınağında aktivdir. Avtomatik Include ayarı yoxdur və hər sınaq təzə context ilə başlayır.

# 18. Heç loading istəməsək nə gəlir?

C# - yalnız Order gətiririk

```
var order = await db.Orders
    .SingleAsync(o => o.Id == 42);

Console.WriteLine(order.CustomerName);
Console.WriteLine(order.Items.Count);
Console.WriteLine(
    db.Entry(order).Collection(o => o.Items).IsLoaded);
```

Yoxlanmış nəticə - lazy bağlıdır

```
Aysel
0
False
SQL: 1
```

Baza Orders-dən 42-ni qaytardı, EF Order obyektini hazırladı. Bu sorğuda OrderItems istənməyib. Sinifdəki `new List<OrderItem>()` boş siyahı yaratdığı üçün Items.Count 0 görünür. **Bazadakı iki məhsul itməyib; sadəcə bu obyektin Items-inə gətirilməyib.**

`Count` burada mövcud kolleksiyadakı element sayıdır. `db.Entry(order)` EF-in həmin obyektlə bağlı idarəetmə məlumatına çıxış verir. `Collection(o => o.Items)` Items əlaqəsini seçir. `IsLoaded` EF-in onu yüklənmiş sayıb-saymadığını bildirir.

False yüklənmiş sayılmadığını göstərir. True və False məntiqi doğru və yanlış dəyərlərdir.

Əvvəl bu context-də məhsul obyektləri alınmış olsaydı nəticə fərqlənə bilərdi. Onu ayrıca tracking hissəsində göstərəcəyik. İndi əsas fakt aydındır: **Order obyekti var, amma Items bazadan oxunmayıb.**

# 19. Eager: məhsulların lazım olduğunu əvvəlcədən de

**Eager loading** əlaqəli məlumatı əsas sorğu ilə birlikdə tələb etməkdir. Burada bunu Include ilə ifadə edirik:

C# - eager sınağı

```
var query = db.Orders.Include(o => o.Items);
// SQL: 0

var order = await query.SingleAsync(o => o.Id == 42);
// SQL: 1, Order və onun 2 məhsulu artıq gəlib.

Console.WriteLine(order.Items.Count);
// 2; SQL yenə 1-dir.
```

`Include(o => o.Items)` “alacağın sifarişlərin Items əlaqəsini də doldur” tələbidir. Sorğunu yalnız qurur; icranı SingleAsync başladır.

SingleAsync-dən sonrakı vəziyyət belədir:

C# obyektlərinin məntiqi görünüşü

```
order: Id = 42, CustomerName = Aysel
  Items[0]: Cay
  Items[1]: Tort
```

Yəni Items.Count oxunanda məhsulları gətirmək üçün əlavə SQL lazım deyil. Onlar artıq gəlib.

Bizim bu ayarlardakı sınağımız bir SQL işlədib. Sonradan göstərəcəyimiz split query eyni eager tələbini bir neçə SQL ilə həyata keçirə bilər. Ona görə eager-in əsas mənası **əlaqəni əvvəlcədən sorğuda istəməkdir**; SQL-in sayını ayrıca ölçürük.

**Əgər ekranda sifarişlə məhsullar mütləq göstəriləcəksə**, bu tələbi sorğunun özündə görmək rahatdır.

# 20. Eager SQL-də əlaqə necə tapılır?

Eager sınağındakı SQL iki cədvəli uyğun açarlar vasitəsilə əlaqələndirir. Məntiqini kiçik SQL ilə görək:

SQL - başa düşmək üçün sadələşdirilib

```
SELECT o.Id, o.CustomerName, i.Id, i.ProductName
FROM Orders AS o
LEFT JOIN OrderItems AS i ON o.Id = i.OrderId
WHERE o.Id = 42;
```

`AS o` və `AS i` cədvəllərə qısa ad verir. `JOIN` cədvəl sətirlərini uyğunlaşdırır. `ON` uyğunlaşma şərtidir: məhsulun OrderId-si sifarişin Id-sinə bərabər olmalıdır.

**LEFT JOIN** soldakı sifarişi, sağda məhsul tapılmasa da nəticədə saxlamağa imkan verir. Beləliklə, məhsulu olmayan sifariş də itmir.

| Sifariş Id | Müştəri | Məhsul |
|---|---|---|
| 42 | Aysel | Cay |
| 42 | Aysel | Tort |

SQL nəticəsində sifariş dəyərləri iki sətirdə təkrarlanır, çünki iki uyğun məhsul var. EF bizim tracking nümunəmizdə bunu **bir Order obyekti və iki OrderItem obyekti** kimi qurur.

SQL sətrinin sayı ilə C#-dakı sifariş obyektinin sayını eyniləşdirmə. Faktiki SingleAsync əmri əlavə məhdudlaşdırıcı hissələr də daşıya bilər; yuxarıdakı SQL əlaqənin məntiqini göstərir.

# 21. Explicit: məhsulları ayrıca əmrlə istə

**Explicit loading** əlaqəni yükləmək üçün açıq şəkildə ayrıca əmr yazmaqdır. Əvvəl Order gəlir, sonra lazım olsa Items gətirilir:

C# - explicit sınağı

```
var order = await db.Orders
    .SingleAsync(o => o.Id == 42);
// SQL: 1; Items hələ gətirilməyib.

await db.Entry(order)
    .Collection(o => o.Items)
    .LoadAsync();
// SQL: 2; indi Items-də 2 məhsul var.

Console.WriteLine(order.Items.Count); // 2
// SQL yenə 2-dir.
```

LoadAsync “bu konkret order obyektinin Items əlaqəsini yüklə” deyir. EF order.Id-nin 42 olduğunu bildiyi üçün məhsul sorğusuna bu şərti qoyur:

SQL - ikinci əmrin sadələşdirilmiş mənası

```
SELECT Id, OrderId, ProductName
FROM OrderItems
WHERE OrderId = 42;
```

Burada əlavə SQL-i Items.Count başlamadı. **Əlavə SQL-i LoadAsync sətri başladı.** Count oxunanda artıq gəlmiş siyahının sayı alınır.

Məsələn, sifarişi gətirəndən sonra iş qaydasına əsasən məhsulların lazım olub-olmadığını qərarlaşdırırsansa bu qərarın altına LoadAsync yaza bilərsən. Əlaqə artıq yüklənmiş sayılırsa eyni LoadAsync çağırışını təkrarlamaq bu sınaqda yeni SQL yaratmır; bu, məcburi yeniləmə əmri deyil.

# 22. Collection və Reference nəyi seçir?

Əlaqə çoxlu obyektə gedirsə Collection, tək obyektə gedirsə Reference yazırıq. Bunlar ayrıca loading növləri deyil; explicit əmrdə hansı əlaqəni seçdiyimizi bildirir.

C# - Order -> çoxlu OrderItem

```
await db.Entry(order)
    .Collection(o => o.Items)
    .LoadAsync();
```

C# - OrderItem -> tək Order; ayrıca təzə context

```
var item = await db.OrderItems
    .SingleAsync(i => i.Id == 101);

await db.Entry(item)
    .Reference(i => i.Order)
    .LoadAsync();

Console.WriteLine(item.Order.CustomerName); // Aysel
```

İkinci nümunədə əvvəl məhsul sətri alınır. Onun OrderId-si 42-dir. Sonra həmin sifariş ayrıca gətirilir. Cəmi iki SQL işləyir.

Yalnız məhsulların sayı lazımdırsa obyektləri gətirmədən hesablamaq da olar:

C# - ayrıca təzə context-də Order alındıqdan sonra

```
var count = await db.Entry(order)
    .Collection(o => o.Items).Query().CountAsync();
```

`Query()` əlaqə üçün sorğu qurmağa çıxış verir. CountAsync bazada sayı hesablayır və 2 qaytarır. **Items obyektlərini yükləmir.** Bu sınaqda IsLoaded False qalır. Bir say almaqla əlaqəni obyektlərlə doldurmaq ayrı işlərdir.

# 23. Lazy üçün nəyi aktivləşdirmək lazımdır?

**Lazy loading** yüklənməmiş əlaqə oxunanda onu gətirə bilən mexanizmdir. EF Core-da öz-özünə hər layihədə aktiv olmur. Bu dərsdə proxy variantını istifadə edirik.

**Proxy** EF-in bizim sinifdən törədərək yaratdığı xüsusi obyekt variantıdır. O, navigation oxunuşuna müdaxilə edib yükləməni başlada bilir. **Törəmə sinif** başqa sinfin imkanlarını götürüb genişləndirən sinifdir.

C# - Program.cs-də lazy rejimi üçün

```
builder.UseLazyLoadingProxies();
```

Bu ayar üçün `Microsoft.EntityFrameworkCore.Proxies` paketi lazımdır. **Paket** layihəyə əlavə edilən hazır kitabxanadır. Tam layihədə versiyası göstərilib.

C# - modeldə navigation belə yazılıb

```
public virtual ICollection<OrderItem> Items { get; set; }
    = new List<OrderItem>();
```

`virtual` törəmə sinfin həmin xassənin davranışını dəyişə bilməsinə imkan verir. Proxy bu imkandan istifadə edir. Proxy üçün sinif də törədilə bilən olmalıdır; bu nümunədə public-dır və sealed deyil. **Sealed** sinifdən törəmə yaratmağı qadağan edən açar sözdür.

**Getter** xassənin oxunarkən işləyən hissəsidir. Sonrakı səhifədə Items getter-i əlavə SQL başladacaq. Təkcə virtual yazmaq bu mexanizmi aktiv etmir; paket, ayar və uyğun obyekt də lazımdır. EF-in proxy-siz başqa lazy mexanizmi də var; bu nümunə proxy variantına aiddir.

# 24. Lazy-də SQL-i hansı sətir başladır?

C# - lazy aktivdir, context təzədir

```
var order = await db.Orders
    .SingleAsync(o => o.Id == 42);
// SQL: 1; yalnız Order gəlib.

var count = order.Items.Count;
// Items getter-i əlaqəni gətirir; SQL: 2.

Console.WriteLine(count); // 2
var again = order.Items.Count;
// Əlaqə artıq yüklənib; SQL yenə 2-dir.
```

İlk Items oxunuşunda proxy yoxlayır ki, əlaqə hələ yüklənməyib. Sonra OrderId = 42 olan məhsulları ayrıca SQL ilə gətirir və Items-i doldurur. Ardınca Count həmin iki obyektin sayını qaytarır.

Burada kodda LoadAsync görmədik, amma baza yenə oxundu. **Gizli görünən əlavə işi Items-in oxunuşu başlatdı.** Buna görə adi xassə oxunuşunun baha baza əməliyyatı olub-olmadığını ayardan asılı olaraq düşünmək lazımdır.

Items birbaşa foreach ilə oxunsa da getter-ə müraciət edilir. Bu nümunədə təkcə Items obyektinə keçid də yükləməni başlada bilər; Count-un özünü ayrıca SQL COUNT əmri sayma.

Lazy sorğunun işləməsi üçün istifadə etdiyi context bağlanmamış olmalıdır. `Dispose` context-in istifadəsini bitirib resurslarını azad etməkdir. Context bağlandıqdan sonra hələ yüklənməmiş əlaqəni oxumaq sınağımızda xəta verir.

Əsas Order sorğusunda await yazmağımız Items getter-ini asinxron etmir. Bu proxy nümunəsində getter adi sinxron yükləmə edir. Asinxron yükləmə üzərində açıq nəzarət lazımdırsa LoadAsync variantını düşün.

# 25. Üç variantı bir zaman xəttində müqayisə et

Hər sınaqda eyni 42 nömrəli sifarişi alırıq. Ayarlar əvvəlki səhifələrdəki kimidir və başlanğıcda əlaqələr yüklənməyib.

| Addım | Eager | Explicit | Lazy |
|---|---|---|---|
| Sorğunu qururuq | Include var; SQL 0 | Include yox; SQL 0 | Include yox; SQL 0 |
| SingleAsync bitir | Order + Items hazır; SQL 1 | Order hazır; SQL 1 | Order hazır; SQL 1 |
| Sonrakı addım | Items.Count | LoadAsync, sonra Count | Items.Count |
| Əlavə SQL-i başladan | Əlavə SQL yoxdur | LoadAsync | İlk Items getter-i |
| Yekun Items / SQL | 2 / 1 | 2 / 2 | 2 / 2 |

**Explicit və lazy-nin burada SQL sayı eynidir, amma nəzarət nöqtəsi fərqlidir.** Explicit-də yükləmə əmrini kodda görürsən. Lazy-də uyğun xassə oxunuşu bu işi başladır.

**Eager və deferred bir yerdə ola bilər.** Include yazıb sonra ToListAsync gözləmək bunun nümunəsidir. Eager əlaqənin tələb olunma qaydasıdır, deferred əsas sorğunun nəticəsinin nə vaxt oxunmasıdır.

**IEnumerable və lazy də eyni anlayış deyil.** IEnumerable elementləri oxumaq müqaviləsidir. Lazy loading isə entity əlaqəsinə müraciətdə ayrıca məlumat gətirmək mexanizmidir.

Özün yoxla: `var q = db.Orders.Include(o => o.Items);` sətrində SQL sayı neçədir? **0.** Items-in lazım olduğunu dedik, amma nəticəni hələ istəmədik.

# 26. Boş Items bazada məhsul yoxdur deməkdirmi?

İki fərqli vəziyyəti yanaşı görək. Lazy bağlıdır, context-lər təzədir:

C# - Order 42, əlaqə yüklənməyib

```
var order = await db.Orders.SingleAsync(o => o.Id == 42);
// Items.Count: 0; IsLoaded: False
```

C# - Order 44, əlaqə yüklənib, amma məhsul yoxdur

```
var order = await db.Orders.Include(o => o.Items)
    .SingleAsync(o => o.Id == 44);
// Items.Count: 0; IsLoaded: True
```

42 üçün bazada iki məhsul var, amma gətirməmişik. 44 üçün əlaqəni istəmişik, baza uyğun məhsul tapmayıb. İkisində də Count 0-dır, səbəbləri fərqlidir.

IsLoaded kömək edir, amma “bazadakı bütün mümkün sətirlər gəlib” zəmanəti deyil. Filtrli Include ilə əlaqənin bir hissəsini yükləyə bilərsən və EF onu yenə yüklənmiş saya bilər.

Digər tərəfdən IsLoaded False olsa da əvvəl yüklənmiş obyektlər hesabına Items qismən dolmuş ola bilər. Bu davranışı növbəti tracking nümunəsində açacağıq.

**Düzgün sual:** “Items-də neçə obyekt var?” ilə yanaşı “bu əlaqəni hansı sorğu ilə, hansı filtr və ayarla yükləmişəm?”

Bu sual yalnız texniki detal deyil. “Sifariş boşdur” qərarı verən iş qaydası yüklənməmiş boş siyahıya əsaslansa yanlış nəticə çıxara bilər.

# 27. N+1: bir siyahı niyə dörd SQL yaradır?

N+1 çox vaxt əvvəl bir əsas siyahı sorğusu, sonra hər elementin əlaqəsi üçün ayrıca sorğu işləməsi deməkdir. N əsas siyahıdakı elementlərin sayıdır.

C# - lazy aktivdir

```
var orders = await db.Orders.OrderBy(o => o.Id)
    .ToListAsync();
// 3 Order gəldi; SQL: 1.

foreach (var order in orders)
    Console.WriteLine(order.Items.Count);
```

Yoxlanmış nəticə

```
2
1
0
Cemi SQL: 4
```

Birinci SQL üç sifarişi gətirdi. Sonra 42 üçün məhsullar, 43 üçün məhsullar və 44 üçün məhsullar ayrıca istəndi. 44-də məhsul olmaması sorğunu aradan qaldırmır: bazadan soruşmadan bunu bilmirik.

**1 + 3 = 4 SQL.** Yüz sifarişdə bu qayda ilə 101 SQL yarana bilər. Əmrlərin sayının artması gecikməni artırır; baza ilə əlaqə uzaqdan qurulursa hər əlavə gediş-gəliş də vaxt aparır.

Eyni dövrdə hər Order üçün `Collection(...).LoadAsync()` yazsaq explicit üsul da bu nümunədə dörd SQL yaradır. Problem yalnız lazy sözü deyil, **hər element üçün ayrıca sorğu qurulmasıdır**.

Əsas sorğunu ToListAsync ilə əvvəldən icra etmək bu problemi həll etmir. Sifariş siyahısı hazırdır, amma onun əlaqələri hələ hazır olmaya bilər.

# 28. Bütün məhsullar lazımdırsa, yoxsa yalnız say?

Məhsulların adlarını göstərmək lazımdırsa əlaqəni əvvəlcədən istəyə bilərik:

C# - bu sınaqda bir SQL

```
var orders = await db.Orders.Include(o => o.Items)
    .OrderBy(o => o.Id).ToListAsync();

foreach (var order in orders)
    Console.WriteLine(order.Items.Count);
```

Amma ekrana yalnız müştəri adı və məhsul sayı çıxacaqsa bütün OrderItem obyektlərini gətirmək vacib deyil:

C# - projection; bu sınaqda bir SQL

```
var rows = await db.Orders.OrderBy(o => o.Id)
    .Select(o => new
    {
        o.Id,
        o.CustomerName,
        ItemCount = o.Items.Count
    })
    .ToListAsync();
```

**Select** nəticədə hansı məlumatları istədiyimizi seçir. Buna **projection** deyilir. `new { ... }` ayrıca ad vermədən nəticə quruluşu yaratmaqdır. Burada Id, CustomerName və ItemCount alınır. Sayı baza hesablayır; Items kolleksiyasını doldurmuruq.

Bu `Items.Count`, əvvəlki lazy nümunəsindəki ilə eyni yerdə deyil. **Burada hələ SQL-ə çevrilən sorğunun içindədir.** Lazy nümunəsində isə artıq alınmış order obyektinin Items-i oxunurdu.

**DTO** cavabda ötürmək istədiyimiz məlumatları daşıyan ayrıca quruluşdur. Belə nəticəni adlandırılmış DTO sinfi ilə də qaytara bilərik. API-də cavabı hazırlayarkən nə gətirdiyimiz aydın olur.

# 29. Tracking loading-dən ayrı bir məsələdir

**Tracking** EF-in gətirilmiş entity-lər barədə dəyişiklik məlumatını context-də saxlamasıdır. Sonra SaveChangesAsync çağıranda hansı dəyişiklikləri bazaya yazmalı olduğunu müəyyən edə bilər.

C# - dəyişiklik izlənir

```
var order = await db.Orders.SingleAsync(o => o.Id == 42);
order.CustomerName = "Aysel Yeni";
await db.SaveChangesAsync();
```

Burada CustomerName dəyişməsi bazaya yazılır. Sadəcə dəyişəni dəyişmək dərhal SQL UPDATE işləmir; saxlanma əmri bunu edir. **UPDATE** mövcud sətrin məlumatını dəyişən SQL əməliyyatıdır.

C# - ayrıca sınaqda dəyişiklik izlənmir

```
var order = await db.Orders.AsNoTracking()
    .Include(o => o.Items).SingleAsync(o => o.Id == 42);
order.CustomerName = "Aysel Yeni";
await db.SaveChangesAsync();
```

İkinci sınaqda Order və Items gətirilir, amma bu dəyişiklik sadəcə öz-özünə SaveChanges ilə yazılmır. AsNoTracking “əlaqələri gətirmə” deyil, **bu sorğunun entity-lərinin dəyişikliklərini izləmə** tələbidir.

Yalnız oxuma üçün tracking yükünü azaltmaq faydalı ola bilər. Dərsdəki EF Core 10.0.9 proxy sınağında AsNoTracking ilə alınan obyekt lazy loading edə bilir. Ona görə “no tracking varsa lazy mütləq işləmir” qaydası qurma.

Context bir sorğunun nəticəsini izləyə bilər, amma bu, növbəti eyni LINQ sorğusunun bazaya getməyəcəyinə zəmanət vermir. Əvvəlki təkrar foreach sınağında bunu ölçdük.

# 30. Əvvəlki sorğu Items-i niyə dəyişə bilər?

EF eyni context-də izlənən əlaqəli obyektləri xarici açarlarına görə bir-birinə bağlaya bilir. Buna **relationship fixup** deyilir.

C# - lazy bağlıdır, əvvəl Order gətirilir

```
var order = await db.Orders.SingleAsync(o => o.Id == 42);

// Sonra onun məhsullarından yalnız biri gətirilir.
var item = await db.OrderItems
    .SingleAsync(i => i.Id == 101);

Console.WriteLine(order.Items.Count); // 1
Console.WriteLine(
    db.Entry(order).Collection(o => o.Items).IsLoaded);
// False; cəmi SQL: 2
```

OrderId = 42 olan item alınanda EF onun context-dəki Order 42-yə aid olduğunu görür və Items-ə bağlayır. **Include yazmadıq, amma Items-də bir obyekt göründü.** Digər məhsul isə hələ gəlməyib.

Sonra `Collection(o => o.Items).LoadAsync()` yazsaq qalan əlaqəli məlumat gətirilir. Bu sınaqda Items 2, SQL 3 olur.

Bu səbəbdən öyrənmə nümunələrində hər təcrübəni təzə DbContext ilə başladırıq. Əks halda əvvəlki sorğunun təsirini yeni loading üsulunun işi ilə qarışdırmaq asandır.

“Include olmasa Items həmişə boşdur” cümləsi ümumi qayda deyil. Düzgün başlanğıc şərti belə idi: **lazy bağlı, avtomatik Include yox, context təzədir və əlaqəli obyektlər əvvəl alınmayıb.**

# 31. Filtrli Include nəyə görə tam siyahı demək deyil?

C# - təzə context-də yalnız bir məhsul istənir

```
var order = await db.Orders
    .Include(o => o.Items.Where(i => i.Id == 101))
    .SingleAsync(o => o.Id == 42);

Console.WriteLine(order.Items.Count); // 1
Console.WriteLine(
    db.Entry(order).Collection(o => o.Items).IsLoaded);
// True; SQL: 1
```

Bu **filtered Include**, yəni filtrli əlaqə yükləməsidir. Order 42-nin iki məhsulu var, amma biz yalnız Id-si 101 olanı istədik. EF bu əlaqəni yüklənmiş sayır. Sonra LoadAsync çağırmaq bu sınaqda qalan məhsulu avtomatik gətirmir.

Eyni tracking context-də 102 əvvəl gətirilibsə fixup nəticəyə onu da bağlaya bilər. O halda filtrli Include nəticəsində belə iki məhsul görə bilərsən. **SQL filtri ilə context-də artıq izlənən obyektlərin təsirini ayırmaq lazımdır.**

Yalnız müəyyən hissəni göstərən oxu nəticəsi lazımdırsa təzə context və ya uyğun AsNoTracking sorğusu əvvəlki tracking təsirini azaltmağa kömək edir. Projection ilə cavabın quruluşunu birbaşa seçmək də aydın ola bilər.

Bu iki filtr fərqlidir:

C# - yalnız 42 nömrəli sifarişi seçir

```
db.Orders.Where(o => o.Id == 42);

// Seçilmiş sifarişlərin daxilindəki Items-i daraldır.
db.Orders.Include(o => o.Items.Where(i => i.Id == 101));
```

Birinci filtr əsas sifarişləri, ikinci isə yüklənəcək əlaqəli elementləri seçir.

# 32. Split query dördüncü loading növüdürmü?

Xeyr. Bu, əlaqələri istədiyimiz sorğunun SQL əmrlərinə necə bölünməsi ilə bağlı seçimdir. **AsSingleQuery** bir SQL, **AsSplitQuery** uyğun kolleksiya yükləməsini ayrıca SQL-lərə bölməyi tələb edir.

C# - üç sifariş və Items; eager tələbidir

```
var orders = await db.Orders.Include(o => o.Items)
    .AsSplitQuery().OrderBy(o => o.Id).ToListAsync();
```

Bu sınaqda iki SQL işləyir: biri sifarişləri, digəri onların məhsullarını gətirir. **Üç sifariş üçün bir-bir üç məhsul sorğusu işləmir.** Bu nümunə lazy N+1-dəki dörd SQL-dən fərqlidir.

Müqayisə - həmin model və ayarlar üçün

```
Eager, AsSingleQuery: 1 SQL
Eager, AsSplitQuery:  2 SQL
Lazy, her Items-e baxmaq: 4 SQL
```

Niyə bölmək istəyə bilərik? Tutaq ki, ayrıca genişləndirilmiş modeldə Order-un həm iki Items-i, həm üç Payments-i var. **Payment ödəniş obyektidir.** İki kolleksiya eyni səviyyədə JOIN ediləndə SQL nəticəsində 2 x 3 = 6 kombinasiya yarana bilər. Buna cartesian explosion, yəni sətir sayının kombinasiyalarla böyüməsi deyilir.

İki kolleksiya üçün yoxlanmış genişləndirilmiş sınaqda single 1, split 3 SQL işlədir. Az təkrarlanan məlumat üstünlük ola bilər; əlavə əmrlər isə əlavə gediş-gəliş və arada məlumat dəyişməsi ehtimalı gətirir. Həmişə daha sürətli olacağına zəmanət yoxdur.

# 33. ThenInclude və səhifələmə nəyi dəyişir?

**Include** Order-dan başlayan əlaqəni seçir. **ThenInclude** əvvəl seçilmiş əlaqənin növbəti pilləsini seçir. Tutaq ki, ayrıca genişləndirilmiş modeldə OrderItem-in Product adlı katalog məhsulu var:

C# - genişləndirilmiş modelin fraqmenti

```
var orders = await db.Orders
    .Include(o => o.Items)
        .ThenInclude(i => i.Product)
    .ToListAsync();
```

Yol belədir: **Order -> Items -> Product**. İkinci müstəqil əlaqə üçün isə yenidən Order-dan başlayan Include yazılır. Product və Payments əvvəlki iki sinifli əsas modelə daxil deyil; bu fraqmenti ora olduğu kimi yapışdırmaq olmaz.

Bu əlavə modellərin tam tərifləri və sınağını Git-də `Advanced_Demo` qovluğuna yerləşdirmişik. Beləliklə, yeni adın arxasındakı quruluşu da görə bilərsən.

Çox sifariş olanda hamısını göstərmək əvəzinə hissə-hissə almağa **səhifələmə - pagination** deyilir:

C# - əsas model; ilk 2 sifariş və onların məhsulları

```
var page = await db.Orders.OrderBy(o => o.Id)
    .Take(2).Include(o => o.Items)
    .AsSplitQuery().ToListAsync();
```

`Take(2)` ilk iki əsas nəticəni seçir. `Skip(n)` əvvəlki n nəticəni ötürmək üçündür. Burada Id-yə görə sabit, təkrarsız sıra verdik; 42 və 43 gəlir. Ayarlardakı sınaqda iki SQL işləyir. **Sifariş sayını məhdudlaşdırmaqla hər sifarişin Items-ni filtrləmək ayrı işlərdir.**

# 34. Hansı kodu harada seçərdim?

**Sifarişin detalları səhifəsi:** məhsullar da həmişə göstərilirsə Include ilə istə, ya da göstəriləcək sahələri Select ilə birbaşa cavab quruluşuna seç.

**Yalnız say və müştəri adı olan siyahı:** Select daxilində ItemCount hesabla. Bütün məhsul obyektlərini sırf saymaq üçün gətirmə.

**Məhsulların lazım olub-olmadığı Order gələndən sonra müəyyənləşir:** qərar verilən yerdə explicit LoadAsync yükləmə anını açıq göstərir.

**Lazy mexanizmi olan layihə:** Items kimi xassə oxunuşunda əlavə SQL ola biləcəyini bil. Dövrlərdə, xəta axtarışında və cavabı mətnə çevirərkən SQL qeydlərini yoxla.

**Serialization** obyektin JSON kimi ötürülən mətn quruluşuna çevrilməsidir. JSON sahə adları və dəyərlərlə məlumat ötürmə formatıdır. Çevirən kod navigation getter-lərini oxuyarsa lazy əlavə SQL işlədə bilər. DTO/projection istifadə edərək cavabı context açıq ikən hazırlamaq bu davranışı aydınlaşdırır.

**ASP.NET Core-da adi HTTP sorğusunun scope-unu framework yaradır; EF yaratmır.** Scope eyni iş üçün obyektlərin paylaşılma və istifadəsinin bitmə sərhədidir. AddDbContext standart olaraq context-i həmin scope daxilində paylaşır. Scope bitdikdən sonra qalan query və ya lazy əlaqəni oxumağa arxalanma.

SQL sayı tək ölçü deyil: gələn sütun və sətir miqdarı, yaddaş və vaxt da vacibdir. Qərarı əvvəlcə hansı məlumatın lazım olduğuna görə ver, sonra real SQL-i ölç.

# 35. İşlək layihəni necə açacaqsan?

Git-də `EF_Core_Loading_Demo` qovluğunu aç. Bu, ayrıca işləyən **Console layihəsidir**: nəticəni terminala yazan proqram. .NET 10 SDK tələb edir. SDK proqramı yığmaq və işə salmaq üçün vasitələr dəstidir.

Terminal - həmin qovluqda ayrı-ayrı işlət

```
dotnet run -- query
dotnet run -- enumerable
dotnet run -- boundary
dotnet run -- eager
dotnet run -- explicit
dotnet run -- lazy
```

`--` işarəsindən sonrakı söz bizim proqramın rejim adıdır. Hər işə salınmada eyni üç sifarişli yeni baza hazırlanır. Əvvəlki işə salınmanın nəticəsi sonrakına daşınmır.

Layihədə Models entity-ləri, CafeContext baza çıxışını, SeedData başlanğıc məlumatını, Program ümumi başlanğıcı saxlayır. QueryLessons isə query/enumerable kimi rejimlərin ayrı kodlarını saxlayır. Hər rejim PDF-də gördüyün hadisəni təkrarlayır.

SQL qeydlərində `Executed DbCommand` həqiqətən işlədilmiş baza əmrini bildirir. Sadəcə ekrana yazılmış ToQueryString mətni belə icra qeydi deyil.

Sonrakı səhifələrdə əsas faylları bütöv veririk. QueryLessons-dəki rejimlərin də tam mənbə kodu Git-də var. Advanced_Demo ayrıca Product/Payment modelidir; əsas iki sinifli nümunəni onun faylları ilə qarışdırma.

# 36. Layihə faylı və DbContext

`.csproj` layihənin hədəfini və asılı olduğu paketləri bildirir. Aşağıdakı versiyalar bu dərsin sınağında istifadə olunub; bütün gələcək versiyalar üçün tələb kimi verilməyib.

EfLoadingDemo.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>
  <ItemGroup>
    <Compile Remove="Advanced_Demo/**/*.cs" />
    <Compile Remove="Loading_Checks/**/*.cs" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Sqlite" Version="10.0.9" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Proxies" Version="10.0.9" />
    <PackageReference Include="SQLitePCLRaw.bundle_e_sqlite3" Version="3.0.5" />
  </ItemGroup>
</Project>
```

CafeContext.cs

```
using Microsoft.EntityFrameworkCore;

public class CafeContext : DbContext
{
    public CafeContext(DbContextOptions<CafeContext> options)
        : base(options)
    {
    }

    public DbSet<Order> Orders => Set<Order>();
    public DbSet<OrderItem> OrderItems => Set<OrderItem>();
}
```

`options` provider və SQL qeydi kimi ayarlardır; konstruktor onları DbContext-ə ötürür. `Set<Order>()` Order üçün DbSet verir. Burada db.Orders-ın bazaya çıxış nöqtəsini necə aldığını görürük.

# 37. Models.cs və əlaqənin qurulması

Models.cs

```
public class Order
{
    public int Id { get; set; }
    public string CustomerName { get; set; } = "";
    public virtual ICollection<OrderItem> Items { get; set; }
        = new List<OrderItem>();
}

public class OrderItem
{
    public int Id { get; set; }
    public int OrderId { get; set; }
    public string ProductName { get; set; } = "";
    public virtual Order Order { get; set; } = null!;
}
```

`int` tam ədəd, `string` mətn tipidir. `{ get; set; }` xassənin oxunub dəyişdirilə bildiyini göstərir. `= ""` başlanğıc mətnini boş qoyur. `new List<OrderItem>()` yeni boş məhsul siyahısı yaradır.

`null!` işarəsi Order obyektini yaratmır və bazaya getmir. Bu yazılış C#-ın null ola bilmə xəbərdarlığını həmin başlanğıc ifadəsi üçün susdurur; runtime-da boş əlaqəni təhlükəsiz etmir. Navigation-u EF yükləyəcək deyə belə yazılıb.

EF bu nümunədə Id, OrderId və navigation adlarından əlaqəni standart qaydaları ilə tanıyır. Daha fərqli modeldə əlaqəni ayrıca ayarla göstərmək lazım ola bilər. `virtual` sözünün proxy lazy loading üçün rolunu 23-cü hissədə gördük.

# 38. SeedData.cs eyni başlanğıcı yaradır

SeedData.cs

```
using Microsoft.EntityFrameworkCore;

public static class SeedData
{
    public static async Task CreateAsync(
        DbContextOptions<CafeContext> options)
    {
        await using var db = new CafeContext(options);
        await db.Database.EnsureCreatedAsync();

        db.Orders.AddRange(
            new Order { Id = 42, CustomerName = "Aysel" },
            new Order { Id = 43, CustomerName = "Murad" },
            new Order { Id = 44, CustomerName = "Leyla" });

        db.OrderItems.AddRange(
            new OrderItem
            { Id = 101, OrderId = 42, ProductName = "Cay" },
            new OrderItem
            { Id = 102, OrderId = 42, ProductName = "Tort" },
            new OrderItem
            { Id = 103, OrderId = 43, ProductName = "Qehve" });

        await db.SaveChangesAsync();
    }
}
```

**Seed data** sınağın başlanğıc məlumatıdır. EnsureCreatedAsync sxemi, yəni cədvəl quruluşunu yaradır. AddRange obyektləri əlavə olunacaq kimi context-ə verir; SaveChangesAsync onları bazaya yazır.

Bu hazırlıq üçün işləyən SQL-lər sonra sayğacdan təmizlənir. Dərsdə ölçdüyümüz rəqəmlər hazırlığa deyil, yoxladığımız oxu əməliyyatına aiddir.

# 39. Program.cs proqramın giriş nöqtəsidir

Program.cs

```
using Microsoft.Data.Sqlite;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Diagnostics;

var mode = args.Length == 0 ? "eager" : args[0];
using var connection = new SqliteConnection("Data Source=:memory:");
await connection.OpenAsync();
var sql = new List<string>();

var builder = new DbContextOptionsBuilder<CafeContext>()
    .UseSqlite(connection)
    .LogTo(sql.Add, new[] { RelationalEventId.CommandExecuted });
if (mode == "lazy") builder.UseLazyLoadingProxies();
var options = builder.Options;

await SeedData.CreateAsync(options);
sql.Clear();

await using var db = new CafeContext(options);
if (await QueryLessons.RunAsync(mode, db, sql)) return;
if (mode is not ("eager" or "explicit" or "lazy"))
    throw new ArgumentException("Unknown lesson: " + mode);
Order order;
if (mode == "eager")
    order = await db.Orders.Include(o => o.Items)
        .SingleAsync(o => o.Id == 42);
else
    order = await db.Orders.SingleAsync(o => o.Id == 42);

Console.WriteLine("Once SQL: " + sql.Count);
if (mode == "explicit")
    await db.Entry(order).Collection(o => o.Items).LoadAsync();

Console.WriteLine("Items: " + order.Items.Count);
Console.WriteLine("Sonra SQL: " + sql.Count);
foreach (var command in sql) Console.WriteLine(command);
```

# 40. Program.cs-ni sətir qrupları ilə oxuyaq

**args** terminaldan verilən sözlərdir. Heç rejim verilməsə eager seçilir. `SqliteConnection` baza bağlantısı obyektidir. `Data Source=:memory:` bu işə salınma üçün yaddaşda SQLite bazası yaradır. Bu, EF InMemory provider-i deyil; həqiqi SQL mühərrikidir.

Bağlantı proqram ərzində açıq qalır ki, başlanğıc məlumatını yazan context bağlandıqdan sonra yaddaşdakı baza itirilməsin. **Bağlantının ömrü ilə DbContext-in ömrü eyni deyil.** Sonra ayrıca təzə oxu context-i yaradılır.

`UseSqlite(connection)` provider-i seçir. `LogTo(sql.Add, ...)` yalnız tamamlanmış SQL əmrlərinin qeydlərini siyahıya yazır. `sql.Clear()` hazırlıq əmrlərini sayğacdan çıxarır.

QueryLessons seçilmiş sorğu dərsini işlədibsə True qaytarır; Program `return` ilə həmin işə salınmanı bitirir. Əks halda eager, explicit və ya lazy nümunəsinə keçir. Tanınmayan rejimdə səssizcə başqa nümunə işlətmir.

`using` və `await using` blokun ömrü bitəndə obyektin istifadəsini düzgün bitirməyi təmin edir. Birincisi adi, ikincisi dəstəklənən asinxron azadetmə üçündür. Bu, obyekti dərhal kompüter yaddaşından silmək demək deyil.

Lazy rejimində `order.Items.Count` sətrinin adi görünüşünə aldanma: aktiv proxy onun içində əlavə SQL başlada bilir. Eager rejimində həmin əlaqə artıq var. Explicit rejimində isə yükləmə əmrini əvvəlki sətirdə açıq görürük.

# 41. Terminal nəticələrini qabaqcadan təxmin et

Əsas üç loading rejimində ilk üç sətir belədir; ardınca faktiki SQL qeydləri yazılır:

Yoxlanmış çıxışlar

```
eager:    Once SQL: 1   Items: 2   Sonra SQL: 1
explicit: Once SQL: 1   Items: 2   Sonra SQL: 2
lazy:     Once SQL: 1   Items: 2   Sonra SQL: 2
```

Digər rejimlər üçün oxu xəritəsi:

| Rejim | Görəcəyin əsas hadisə |
|---|---|
| query | Qurma və ToQueryString: SQL 0; nəticə Id 43, 1 entity. |
| enumerable / boundary | Qurma: SQL 0; nəticə Id 43, amma 2 entity oxunub. |
| foreach | Dövrdən əvvəl SQL 0, sonra 1. |
| repeat | İki query dövrü: SQL 2; ToList sonra 3; List dövrləri artırmır. |
| scalar | Count 2; Any True; birlikdə SQL 2; Order entity-si 0. |
| asqueryable | Üç Order əvvəl gəlib; yerli Count 1; SQL cəmi 1. |
| translation / local | Birincidə SQL-ə çevrilmə xətası; ikincidə açıq yerli filtr. |
| list-deferred | 3 sonradan əlavə olsa da nəticə 2,3-dür; baza işi yoxdur. |
| async-stream | Qurma SQL 0; await foreach 43 və 44-ü oxuyur; SQL 1. |

Entity sayı query sınaqlarında təzə tracking context-dəki Order qeydləri ilə yoxlanıb. Burada nəticəyə daxil olmayan 44-ün belə enumerable variantında gətirildiyini görmək üçündür. Bu ölçünü bütün sorğular üçün ümumi sətir sayğacı kimi istifadə etmirik.

# 42. Özünü kod üzərindən yoxla

**1.** `var q = db.Orders.Where(o => o.Id == 42);` yazdın. Ayselin obyekti artıq gəlibmi? **Xeyr.** Sorğu təsvirini qurdun. Nəticə hələ oxunmayıb.

**2.** `var q = db.Orders.AsEnumerable();` yazdın. Bütün sifarişlər artıq List-dədir? **Xeyr.** AsEnumerable nə icra, nə də List yaratma əmri idi.

**3.** `await db.Orders.ToListAsync()` bitdi. Sonrakı List.Where filtri bazada işləyəcək? **Xeyr.** Artıq gətirilmiş obyektlər üzərində işləyəcək.

**4.** Include olan IQueryable-ı iki dəfə ToListAsync etsən SQL bir dəfə işləyəcək? **Bu sınaq qaydalarında xeyr.** Hər dəfə əsas sorğunu yenidən icra edirsən. Include nəticə yaddaşı deyil.

**5.** `order.Items.Count` 0-dır. Baza boşdur? **Hələ bilmirik.** Loading ayarını, əvvəlki sorğuları və filtrin olub-olmadığını yoxla.

**6.** `Select(o => new { Count = o.Items.Count })` lazy loading-dir? **Bu EF sorğusu nümunəsində xeyr.** İfadə SQL hesablamasına çevrilir; alınmış obyektin getter-ini oxumaqla eyni hadisə deyil.

**7.** Eager işlətmişik, SQL iki dəfədir. Deməli səhvdir? **Mütləq deyil.** AsSplitQuery və faktiki icra qeydlərinə bax.

Bir nümunə anlaşılmayanda bu ardıcıllıqla yoxla: **mənbə nədir -> görünən tip nədir -> filtr harada yazılıb -> icranı hansı sətir başladır -> hansı entity və əlaqələr artıq gətirilib?**

# 43. Mənbələr və nəticələrin sərhədi

Bu dərsin kod nəticələri ayrıca işlədilib. SQL sayları göstərilən model, provider və ayarlara aiddir. Başqa provider, avtomatik Include, əvvəlki tracking məlumatı və model dəyişikliyi nəticəni dəyişə bilər.

Rəsmi izahları aşağıdakı Microsoft sənədlərindən açа bilərsən:

- EF sorğusunun işləməsi və IQueryable müqaviləsi.
- AsEnumerable və deferred execution.
- Bazada və yerli hesablama və asinxron EF əməliyyatları.
- Eager loading, explicit loading, lazy loading.
- Tracking, səmərəli sorğular, single və split query.

Sorğunun işləməsi səhifəsində bəzi daxili mərhələlərin təsvirinin köhnələ bildiyi bildirilir. Burada daxili icra detallarına dair iddialar əvəzinə göstərilən kodların faktiki nəticələrini əsas götürmüşük.

İşlək əsas layihə və bütün query rejimləri PDF-dəki nümunələri dəyişib təkrarlamaq üçündür. Advanced_Demo ayrıca genişləndirilmiş model və sınaqları saxlayır. Əvvəl əsas iki sinifli nümunələri işlə; sonra əlavə əlaqələrə keç.